Quiz #1 (Part II - Extended): Step-by-Step IEEE 754 Conversion

Total Marks: 120.00

Single-Precision (32-Bit) Floating-Point Encoding & Analysis

Name: **Student ID:** **Score:** / 120.00

> **Instructions:** Complete the step-by-step IEEE 754 conversion worksheets for all four problems. Fill in each boxed step with the appropriate binary sequence, arithmetic intermediate, or bit string. For the conceptual sub-questions, adhere strictly to the marked format (Single Choice vs. Select All That Apply). Calculators are not permitted.

Question 1: Terminating Mixed Positive Decimal

Mark 30.00 out of 30.00

Technique / Concept Hint: Successive division by 2 for integer; successive multiplication by 2 for terminating fractions; normalization shift count $E = \text{exponent}$; biasing formula $E + 127$; mantissa trailing zero padding to 23 bits.

To convert +27.3125 to IEEE 754 floating-point format step by step:

1. Sign bit: ☐

2. Convert to binary:

2.1. Integer 27 → ☐

2.2. Fraction 0.3125 → 0.3125 × 2 = ☐ → ☐ × 2 = ☐ → ☐ × 2 = ☐ → ☐ × 2 = ☐ then the binary fraction is ☐

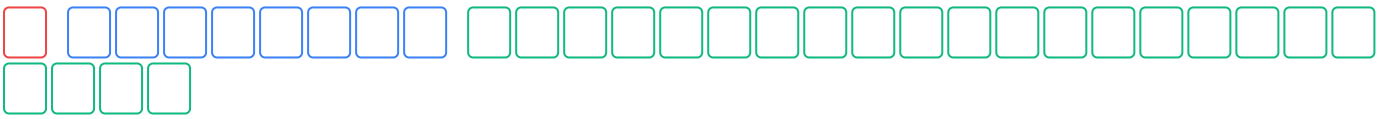
2.3. 27.3125 binary number is ☐ . ☐

3. Normalize: ☐ × 2 ^ ☐

4. Exponent: ☐ + 127 = ☐ → ☐

5. Mantissa (23 bits): ☐

6. IEEE 754 (32-bit layout):



[CONCEPT CHECK — MULTIPLE CHOICE]

Why is the leading 1. omitted when writing the normalized significand into the 23-bit mantissa field?

- ◯ a. Because the standard reserves the leading 1 strictly to denote negative numbers
- ◯ b. Because every non-zero normalized binary number starts with 1, making it redundant and saving 1 bit of storage
- ◯ c. Because single-precision hardware registers can only perform bitwise logic on fractional values
- ◯ d. To allow denormalized numbers to share identical hardware paths without checking flags

Question 2: Non-Terminating Negative Repeating Fraction

Mark 30.00 out of 30.00

Technique / Concept Hint: Fractional multiplication loop detection; right-shift normalization for values < 1 yielding a negative exponent (2^{-k}); rounding to nearest 23 bits (check guard bit 24).

To convert -0.2 to IEEE 754 floating-point format step by step:

1. Sign bit: ☐

2. Convert to binary (fraction only):

0.2 × 2 = 0.4 → ☐ 0.4 × 2 = 0.8 → ☐ 0.8 × 2 = 1.6 → ☐ 0.6 × 2 = 1.2 → ☐ 0.2 × 2 = 0.4 → ☐

(repeats)

→ repeats 0. ☐ ...

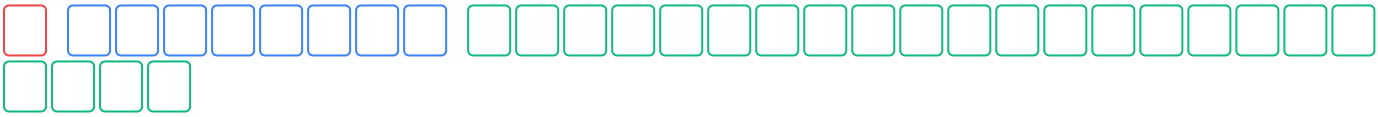
3. Normalize (shift RIGHT until we get the leading 1): ☐ × 2 ^ ☐

4. Exponent: ☐ + 127 = ☐ → ☐

5. Mantissa: ☐ (rounded up to gain 23 bits)

6. IEEE 754 (32-bit layout):

MSB (Bit 31: Sign) | Bits 30-23: Biased Exponent | Bits 22-0: Mantissa (LSB)



[CONCEPT CHECK — SELECT ALL THAT APPLY]

Which statements are TRUE regarding the binary representation of decimal –0.2 and repeating fractions?

- ☐ a. Decimal 0.2 cannot be represented with finite binary digits because its denominator contains prime factors other than 2.
- ☐ b. Rounding the repeating mantissa introduces a small representation error (roundoff error).
- ☐ c. The sign bit of -0.2 is stored as two's complement across all 32 bits.
- ☐ d. Shifting the radix point to the right results in a negative unbiased exponent.

Question 3: Negative Mixed Terminating Decimal

Mark 30.00 out of 30.00

Technique / Concept Hint: Sign-magnitude encoding (sign bit = 1 for negative mixed real); integer repeated division; fractional binary conversion; left-shift normalization.

To convert -13.375 to IEEE 754 floating-point format step by step:

1. Sign bit: ☐

2. Convert to binary:

2.1. Integer 13 → ☐

2.2. Fraction 0.375 → 0.375 × 2 = ☐ → ☐ × 2 = ☐ → ☐ × 2 = ☐

then the binary fraction is ☐

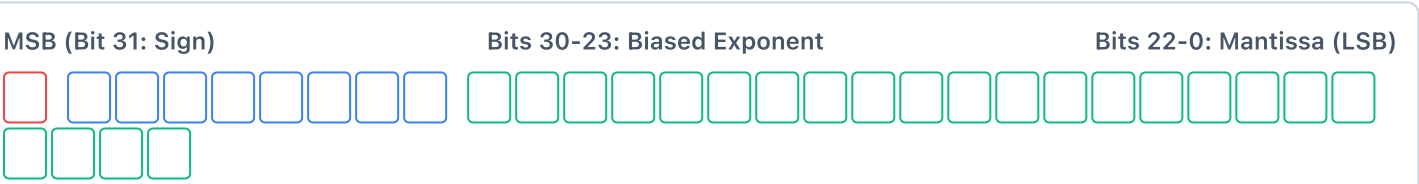
2.3. 13.375 binary number is ☐ . ☐

3. Normalize: ☐ × 2 ^ ☐

4. Exponent: ☐ + 127 = ☐ → ☐

5. Mantissa (23 bits): ☐

6. IEEE 754 (32-bit layout):



[CONCEPT CHECK — MULTIPLE CHOICE]

How does IEEE 754 represent negative numbers in comparison to standard fixed-point integers?

- ◯ a. IEEE 754 inverts all bits and adds 1 across the full 32-bit register (Two's complement)
- ◯ b. IEEE 754 utilizes a sign-magnitude scheme where only bit 31 changes, while exponent and mantissa remain unchanged
- ◯ c. IEEE 754 negates only the exponent field leaving the mantissa unsigned
- ◯ d. IEEE 754 requires negative values to be encoded using excess-64 offset binary

Question 4: Positive Non-Terminating Repeating Fraction

Mark 30.00 out of 30.00

Technique / Concept Hint: Successive fractional doubling; detection of recurring pattern; normalization shift yielding $E = -2$; addition of bias $+127$; rounding bit evaluation.

To convert +0.4 to IEEE 754 floating-point format step by step:

1. Sign bit: ☐

2. Convert to binary (fraction only):

0.4 × 2 = 0.8 → ☐ 0.8 × 2 = 1.6 → ☐ 0.6 × 2 = 1.2 → ☐ 0.2 × 2 = 0.4 → ☐ 0.4 × 2 = 0.8 → ☐

(repeats)

→ repeats 0. ☐ ...

3. Normalize (shift RIGHT until we get the leading 1): ☐ × 2 ^ ☐

4. Exponent: ☐ + 127 = ☐ → ☐

5. Mantissa: ☐ (rounded up to gain 23 bits)

6. IEEE 754 (32-bit layout):

MSB (Bit 31: Sign) | Bits 30-23: Biased Exponent | Bits 22-0: Mantissa (LSB)

☐ ☐☐☐☐☐☐☐☐ ☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐

[CONCEPT CHECK — SELECT ALL THAT APPLY]

Which of the following statements regarding the IEEE 754 exponent field are TRUE?

- ☐ a. The bias value of 127 ensures the stored exponent is always non-negative for all valid normalized numbers.
- ☐ b. An unbiased exponent of -2 produces a stored biased exponent of 125 (`01111101` $_2$).
- ☐ c. An exponent pattern of `00000000` is reserved for subnormal numbers and zeros.
- ☐ d. Storing exponents in biased format eliminates the need for hardware adders during float multiplication.

Instructor Answer Key & Step-by-Step Solutions

Maximum Marks: 120.00

Quiz #1 (Part II - Extended) - Comprehensive Marking Scheme

Question 1 Solution: Converting +27.3125 (30.00 Marks)

```
1. Sign bit: [0]
2. Convert to binary:
  2.1. Integer 27 → [11011]
  2.2. Fraction 0.3125 → 0.3125 × 2 = [0] → [0.625] × 2 = [1] → [0.25] × 2 = [0] → [0.5] × 2 = [1] then the binary fraction is [0101]
  2.3. 27.3125 binary number is [11011].[0101]
3. Normalize: [1.10110101] × 2^[4]
4. Exponent: [4] + 127 = [131] → [10000011]
5. Mantissa: [10110101000000000000000]
6. IEEE 754 (32-bit): MSB [0] [10000011] [10110101000000000000000] LSB
```

Concept Subpart: Correct Option is **b** (In base 2, every non-zero normalized number begins with 1., so omitting it grants an extra bit of precision for free)[cite: 2].

Question 2 Solution: Converting -0.2 (30.00 Marks)

```
1. Sign bit: [1]
2. Convert to binary (fraction only):
  0.2 × 2 = 0.4 → [0]
  0.4 × 2 = 0.8 → [0]
  0.8 × 2 = 1.6 → [1]
  0.6 × 2 = 1.2 → [1]
  0.2 × 2 = 0.4 → [0] (repeats)
  → repeats 0.[001100110011...]
3. Normalize: [1.1001100110011...] × 2^[-3]
4. Exponent: [-3] + 127 = [124] → [01111100]
5. Mantissa: [10011001100110011001101] (rounded up to gain 23 bits)
6. IEEE 754 (32-bit): MSB [1] [01111100] [10011001100110011001101] LSB
```

Concept Subpart: Correct Options are **a, b, d**. Option *c* is false because IEEE 754 uses sign-magnitude, not two's complement[cite: 2].

Question 3 Solution: Converting -13.375 (30.00 Marks)

```
1. Sign bit: [1]
2. Convert to binary:
  2.1. Integer 13 → [1101]
```

```
  2.2. Fraction 0.375 → 0.375 × 2 = [0] → [0.75] × 2 = [1] → [0.5] × 2 = [1] then the
binary fraction is [011]
  2.3. 13.375 binary number is [1101].[011]
3. Normalize: [1.101011] × 2^[3]
4. Exponent: [3] + 127 = [130] → [10000010]
5. Mantissa: [10101100000000000000000]
6. IEEE 754 (32-bit): MSB [1] [10000010] [10101100000000000000000] LSB
```

Concept Subpart: Correct Option is **b** (IEEE 754 strictly adheres to sign-magnitude, where the sign bit is separate from exponent and mantissa)[cite: 2].

Question 4 Solution: Converting +0.4 (30.00 Marks)

```
1. Sign bit: [0]
2. Convert to binary (fraction only):
  0.4 × 2 = 0.8 → [0]
  0.8 × 2 = 1.6 → [1]
  0.6 × 2 = 1.2 → [1]
  0.2 × 2 = 0.4 → [0]
  0.4 × 2 = 0.8 → [0] (repeats)
  → repeats 0.[011001100110...]
3. Normalize: [1.1001100110011...] × 2^[-2]
4. Exponent: [-2] + 127 = [125] → [01111101]
5. Mantissa: [10011001100110011001101] (rounded up to gain 23 bits)
6. IEEE 754 (32-bit): MSB [0] [01111101] [10011001100110011001101] LSB
```

Concept Subpart: Correct Options are **a, b, c**. Option *d* is false because biased exponents still require subtraction/addition operations during multiplication[cite: 2].
